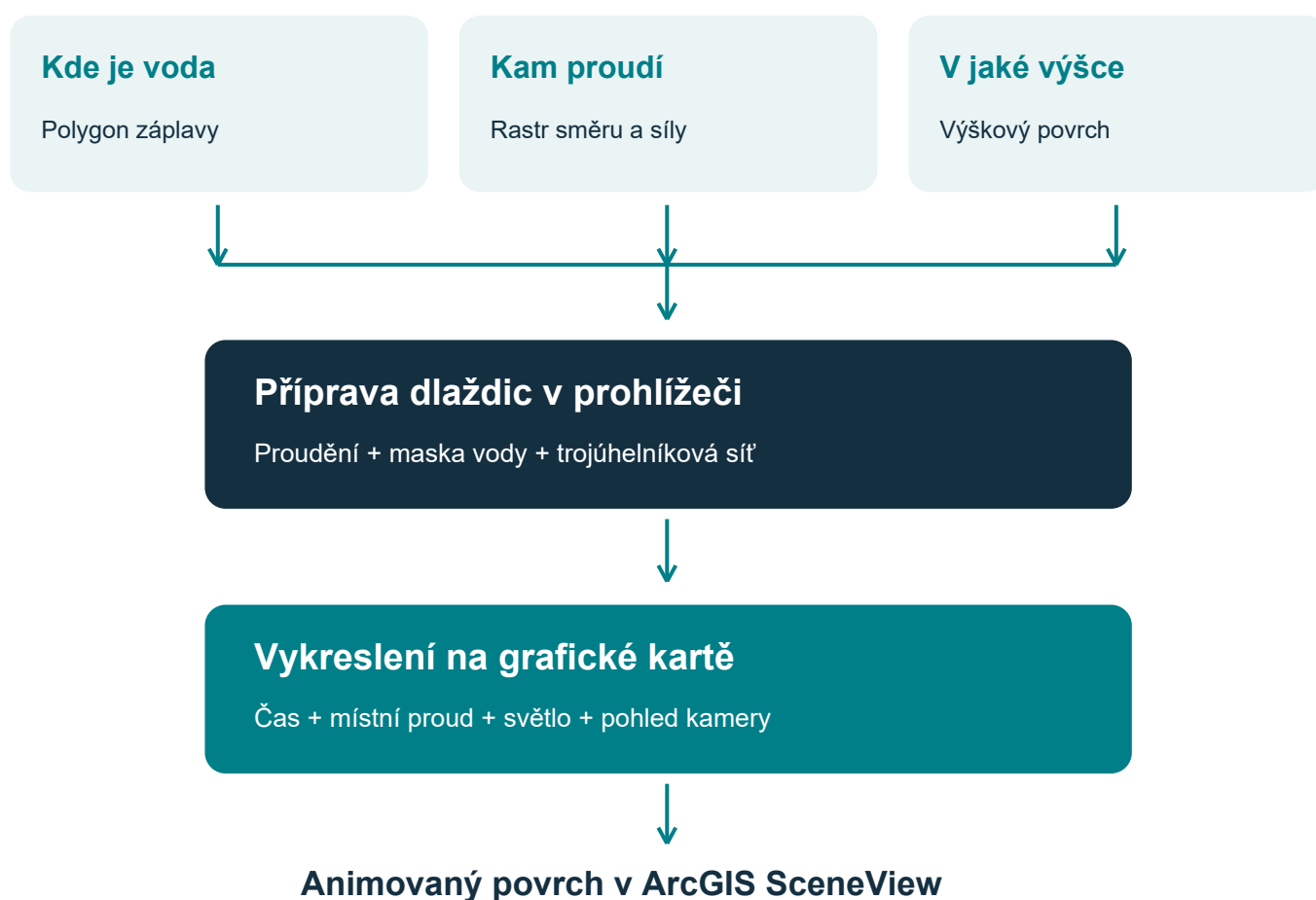


Jak se data promění ve vodu

Renderer přidává do existující 3D scény povrch, jehož kresba se pohybuje podle místního proudění.

Představ si tenkou síť rozprostřenou přes zaplavené území. Síť určuje polohu povrchu. Grafická karta na ní v každém snímku vypočítá drobné sklony a odlesky. Posouváním této kresby ve směru proudu vzniká dojem tekoucí vody.



Co animace znamená: zobrazuje dodané proudové pole. Renderer sám nepočítá vznik povodně, postup záplavy ani novou výšku hladiny.

Tři podklady, tři různé otázky

Oddělení rozsahu, proudění a výšky je zásadní pro správné chování vody.

1. Rozsah zaplavení: kde se smí kreslit

Polygonová vrstva se převede na rastrovou masku. Uvnitř je voda viditelná, mimo polygon se pixel zahodí. Masky respektuje také otvory, například nezaplavené ostrovy. Vyhlazený okraj pomáhá změkčit přechod na břehu.

2. Proudové pole: kterým směrem a jak silně

Dvě pásma rastru obsahují velikost M a směr D . Adaptér je převede na složku k východu a k severu. Teprve tyto složky se mezi pixely interpolují; průměrování samotných úhlů by například mezi 359° a 1° vedlo chybným směrem.

$$\text{východ} = M \cdot \sin(D) \quad \text{sever} = M \cdot \cos(D)$$

D je ve stupních; převod odpovídá konvenci použité v tomto projektu.

Implementace tento výklad označuje jako **flow-from**; převod byl ověřen vůči ArcGIS FlowRendereru. Název není důvodem k automatickému otočení vektoru o 180° . U jiných dat je potřeba ověřit jejich vlastní konvenci.

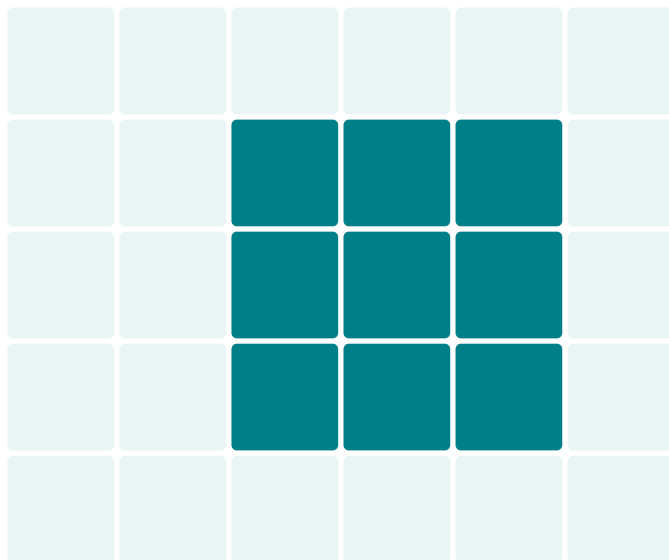
3. Výškový povrch: kam síť umístit

Výšky se vzorkují přes `Ground.queryElevation` ze stejného `Ground` objektu, který používá scéna. Prague demo používá zdroj `teren_hladina5160`. Vrcholy sítě dostanou navíc posun $+0,20$ m, který omezuje probíhání překrývajících se povrchů. Tato výška neudává hloubku vody.

Chybí proudění, ale polygon říká „voda“? Povrch zůstane viditelný a kresba se zklidní a zastaví. Chybějící rychlost nevytvoří díru v záplavě.

Území se načítá po dlaždicích

Aplikace připravuje jen omezenou část území podle kamery a znovu využívá již načtená data.



Dlaždice v okolí pohledu

Okolí / přednačtení

Pevná mřížka má společný geografický počátek. Pohyb kamery mění výběr dlaždic, nikoli jejich polohu nebo fázi animace.

Správce řídí frontu, ruší nepotřebné požadavky a uvolňuje starší data z mezipaměti.

Schéma výběru, nikoli přesný půdorys kamery.

Co obsahuje jedna dlaždice

Velikost území	512 × 512 m
Proudové pole	384 × 384 vzorků + okraj 2 pixely
Maska zaplavení	768 × 768 vzorků + okraj 2 pixely
Výšková síť	192 × 192 buněk, rozdělených na trojúhelníky

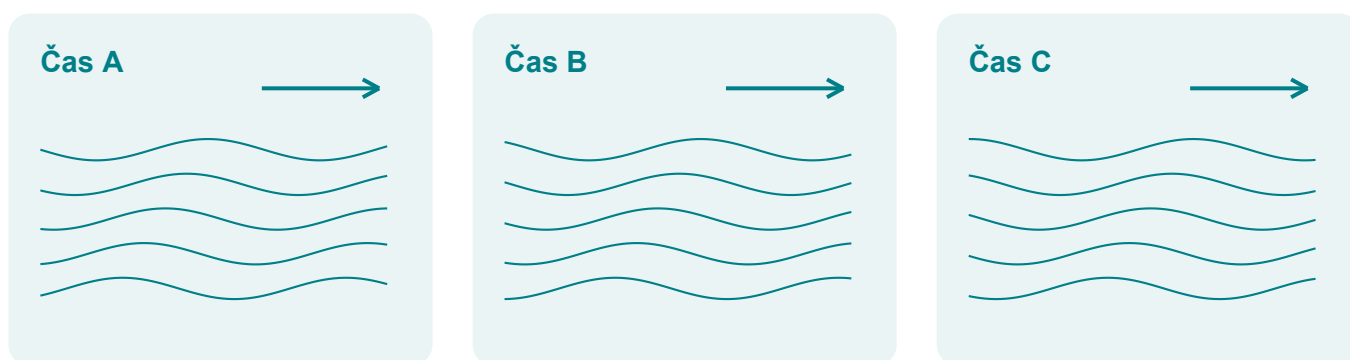
Přesah vzorků za okraj dlaždice pomáhá interpolaci na hranicích. Sousední sítě používají stejné vstupy na společném okraji. Společné souřadnice a čas navazují také kresbu vln.

Výchozí limity: 2 souběžné přípravy, nejvýše 36 požadovaných dlaždic, 96 záznamů v cache a 160 MiB uchovaných CPU polí. Jedna plná dlaždice zabere přibližně 4 MiB na CPU a další 4 MiB na GPU.

Proud posouvá kresbu povrchu

Animace vzniká ve shaderu: malém programu, který grafická karta provádí pro vykreslované pixely.

Pro dané místo shader přečte vektor proudu a spočítá procedurální strukturu pomocí matematického šumu. Nepotřebuje obrázek vln ani video. Kombinuje hrubší a jemnější detail a protahuje jejich strukturu podle místního směru.



Schematický posun detailu po povrchu; nejde o dráhy částic.

Proč se animace při opakování netrhá

Dlouhý nepřetržitý posun vzoru by při proměnlivém proudu způsoboval silné deformace. Renderer proto používá dvě časové fáze posunutí o půl cyklu. Jedna kresba postupně zesiluje, druhá slábne. Ve chvíli návratu fáze na začátek je její váha nulová, takže reset není přímo vidět.

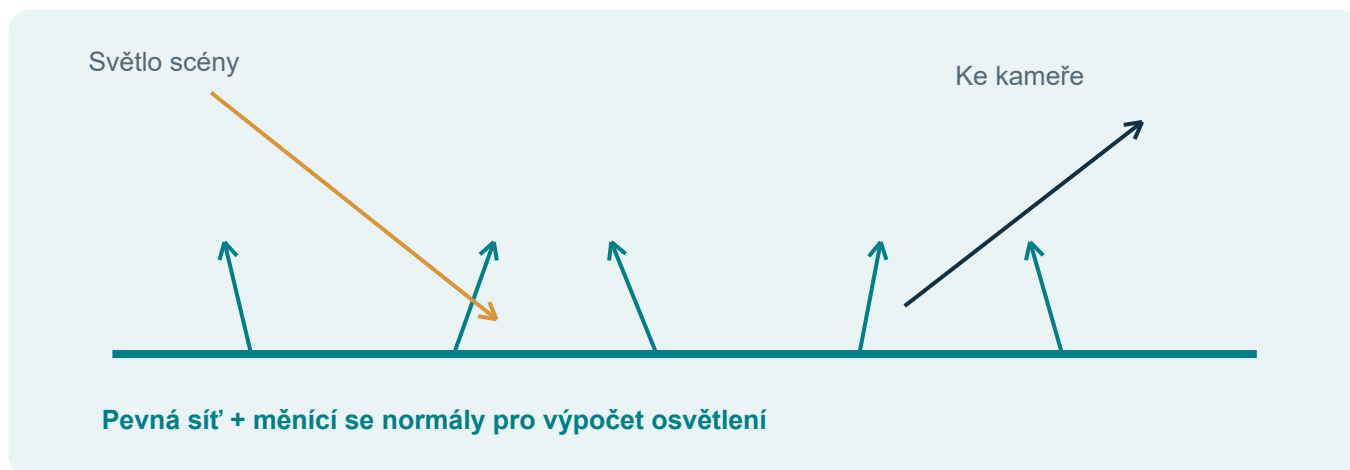
Co se skutečně mění

Z šumu se vypočítá lokální sklon a z něj normála, tedy směr, kterým povrch zdánlivě míří. Změna normály posouvá odlesky. **Vrcholy sítě zůstávají na svém místě:** vlnky představuje osvětlení, nikoli geometrické zvedání a klesání hladiny.

Rychlost je vizuální. Místní vektor řídí směr a míru posunu, ale posuvník flowSpeed není převod na m/s. Jednotky zdrojového proudu Prague demo nejsou zdokumentované; animace není měřítkem doby doplutí.

Světlo vytváří dojem hladiny

Barva každého pixelu závisí na sklonu detailu, osvětlení scény a směru pohledu.



Tři složky výsledné barvy

Barva vody tvoří základ a reaguje na okolní a směrové světlo. **Nádech oblohy** se míchá silněji při šikmém pohledu podél hladiny (Fresnelův efekt). **Světelný odlesk** zvýrazní místa, jejichž normála vhodně směřuje vůči světlu a kameře.

Obloha je zde jednoduchá analytická barva. Renderer neodráží skutečné budovy, mosty ani obsah obrazovky a nepočítá lom světla skrz vodu. Vzdálený jemný detail tlumí, aby méně blikal.

Jak se voda spojí s existující scénou

Vlastní ArcGIS RenderNode vykreslí dlaždice do barevného výstupu scény. Používá hloubkový test, aby respektoval geometrii před vodou, a průhledné prolínání. Masky řídí viditelnost u břehů; nové dlaždice se krátce plynule zobrazují.

Posuvníky dema upravují rychlost animace, jas, odlesky a odstín. Průhlednost lze měnit přes API. Tyto změny nevyžadují nové dotazy na služby ani přestavbu sítě. Pauza zastaví čas animace, kamera může dál pracovat.

Co se děje při používání dema

Data se připravují při změně potřebného území. Samotný pohyb vody pak počítá GPU nad již načtenými dlaždicemi.

1

Otevření dema

Hostitelská aplikace vytvoří WebScene a SceneView. Adaptér najde zdroje, ověří jejich vlastnosti a renderer připraví grafický program.

2

Načtení okolí kamery

Adaptér pro požadované dlaždice získá proudy, polygony a výšky. Připraví pole a síť; renderer je nahraje na GPU.

3

Každý další snímek

Aktualizuje se společný čas, kamera, světlo a parametry vzhledu. GPU vykreslí vodu bez opětovného načítání stejných dat.

4

Přesun nebo ukončení

Při přesunu přibývají potřebné dlaždice a staré se uvolňují podle limitů. `stop()` vyčistí prostředky; `destroy()` ukončí instanci trvale.

Jak číst výsledek: rozsah vody pochází z polygonu, výška ze zadaného povrchu a pohyb z proudového rastru. Jde o vizualizaci jednoho dodaného scénáře, nikoli o výpočet hydrauliky nebo vývoje povodně v čase.

Podklady a hranice této dokumentace

Zpracováno podle zdrojového kódu tohoto projektu: README.md; docs/architecture.md; docs/data-contract.md; examples/prague/scenario.js; src/public/defaults.js; src/data/ArcGISFloodAdapter.js; src/render/TiledWaterRenderNode.js a shaderů `water.frag.glsl`, `field.glsl`. Diagramy jsou vysvětlující schémata, nikoli snímky simulace.

Implementace 0.1.0 používá WebGL2 a globální ArcGIS SceneView. Široké pohledy mají omezené pokrytí dlaždicemi; po rychlém přesunu mohou být dočasně viditelné mezery při načítání.

Zapojení do Prague demo

Aplikace vytváří scénu. Knihovna si vypůjčí její objekty a přidá pouze vlastní vykreslování vody.

Vstupní soubor **examples/prague/main.js** importuje veřejnou knihovnu z dist. Po načtení SceneView zavolá pomocnou funkci se scénářem Prague. Následuje zkrácený výřez skutečného zapojení; obsluha chyb a panel jsou vynechány.

```
import WebScene from '@arcgis/core/WebScene.js';
import SceneView from '@arcgis/core/views/SceneView.js';
import { createFloodWaterFromScenario }
  from '../dist/flood-water-renderer.js';
import { pragueScenario } from './scenario.js';

const scene = new WebScene({
  portalItem: {
    id: pragueScenario.webSceneItemId,
    portal: { url: pragueScenario.portalUrl }
  }
});
const view = new SceneView({ container: 'app', map: scene });
await view.when();
const water = await createFloodWaterFromScenario({
  view, scenario: pragueScenario
});
```

Co udělá createFloodWaterFromScenario()

Vyhledá existující ImageryTileLayer a FeatureLayer podle URL nebo ID vrstvy. Ověří také požadovanou výškovou vrstvu v Ground. Z konfigurace vytvoří **ArcGISFloodAdapter**, z adaptéru a view vytvoří **FloodWaterRenderer**, zavolá start() a vrátí instanci water.

Knihovna nevytváří novou mapu a nemění viditelnost původních vrstev. To dělá panel **WaterControls**: režim Custom water skryje referenční vodu i proudovou vrstvu; kombinovaný režim ponechá FlowRenderer. Přepnutí na Esri water vlastní renderer zastaví.

Kód: examples/prague/main.js; examples/prague/controls.js; src/public/scenario.js

Scénář je oddělený od rendereru

Identita služeb, souřadnicový počátek a parametry načítání jsou uloženy v `examples/prague/scenario.js`.

Scénář / WebScene	prague-5160-v1 / Prague · Q5160 2067a314e0094fc5899a700d78f732cc
Portál	https://iprpraha.maps.arcgis.com
Proudění	MagDir_5160_wgs / ImageServer Vector-MagDir; pásma [0, 1]; flow-from
Rozsah záplavy	ZC_5160 / FeatureServer/0
Výškový zdroj	teren_hladina5160 / ImageServer demResolution: 2; offsetMeters: 0.20
Společný počátek	[14.4125, 50.0815] ve WGS84
Reference aktivity	magnitudeReference: 33.0753930311 Řídí vzhled detailu, nepřevádí jednotky rychlosti.

Co se kontroluje před kreslením

Adaptér načte metadata zdrojů. Proudový rastr musí být vector-magdir, mít dvě pásma a EPSG:4326. Rozsah vody musí tvořit polygony. Ground musí být totožný s `view.map.ground` a mít právě jednu viditelnou výškovou vrstvu. Renderer podporuje globální `SceneView`.

start() není čekání na všechny dlaždice. Promise se splní po ověření zdrojů a inicializaci GPU programu. Událost `ready` znamená první skutečné vykreslení vody. `WeakMap` rezervace brání dvěma aktivním rendererům na jednom view nebo adaptéru.

Kód: `examples/prague/scenario.js`; `src/data/ArcGISFloodAdapter.js`; `src/public/FloodWaterRenderer.js`

Jak vznikne obsah jedné dlaždice

ArcGISFloodAdapter.loadTile() spojuje tři konkrétní dotazy do normalizovaných polí pro vykreslení.

```
// Zjednodušené pořadí v loadTile():
const mask = await getWaterFootprint(tile, signal);
if (mask.empty) return { ...tile, empty: true, bytes: 0 };
const [field, surface] = await Promise.all([
  getVelocityTile(tile, combinedSignal),
  getSurfaceElevation(tile, mask, combinedSignal)
]);
return validateTile({ ...tile,
  field, surface, waterFootprint: mask });
```

A. Polygony se rasterizují v Canvas 2D

queryObjectIds() vybere objekty protínající rozšířený extent. queryFeatures() je načítá po 32, s geometrií ve WGS84. Každý prstenec se převede na pixely canvasu a fill("evenodd") zachová otvory. Alfa kanál canvasu se uloží do Uint8Array; hodnoty 0 až 255 zahrnují i vyhlazený okraj.

B. Proudění se převádí na vektory

fetchPixels(extent, width, height, { bandIds: [0,1], interpolation: "nearest", signal }) získá PixelBlock. Kontrolují se rozměry, extent a masky platnosti. Platný vzorek se uloží jako [east, north, magnitude, 1]; neplatný jako [0, 0, 0, 0]. Záporné a nečíselné velikosti se vyřadí.

C. Výšky se dotazují pro celou síť

queryElevation(Multipoint) získá výšky pro 193 × 193 vrcholů. Chybějící výška pod maskou vody vyvolá chybu dlaždice. Výsledek projde validateTile(): ověří typy polí, konečnost hodnot, platnost indexů a znovu spočítá velikost uložených dat.

Kód: src/data/ArcGISFloodAdapter.js; footprint.js; velocity.js; surface.js; contract.js

Od WGS84 k trojúhelníkům

Geografická mřížka a společný metrický rámec udržují sousední dlaždice prostorově i vizuálně navázané.

TileGrid převádí metry na stupně s $R = 6\,378\,137$ m. Severní měřítko je $R \cdot \pi / 180$, východní navíc násobí $\cos(\text{šířka počátku})$. Dlaždice $[x,y]$ má $\text{worldOffset} = [x \cdot 512, y \cdot 512]$. Jde o regionální aproximaci kolem pevného počátku.

Hraniční vrcholy vznikají z globálních celočíselných indexů, takže sousedé žádají stejné souřadnice. Po přičtení výškového posunu převede `toRenderCoordinates()` body do renderovacího prostoru globální scény. Od každého bodu se odečte počátek dlaždice a relativní XYZ se uloží do Float32.

Data	Výchozí velikost / GPU reprezentace
Proudové pole	$388 \times 388 \times 4$ hodnot Float32 RGBA32F: east, north, magnitude, valid
Maska vody	772×772 hodnot Uint8 R8: míra pokrytí vodou
Vrcholový buffer	$37\,249$ vrcholů $\times 5$ Float32 relativní X, Y, Z, U, V; stride 20 bajtů
Indexový buffer	$221\,184$ indexů Uint16 $73\,728$ trojúhelníků; 2 na každou buňku

Součet těchto polí je **4 192 036 bajtů, přibližně 4,00 MiB**, na jednu plnou dlaždici. Podobný objem se nahraje na GPU. Nezahrnuje interní cache ArcGIS, dočasná pole ani režii ovladače.

UV a přesahy: logické UV má jihozápad (0,0), severovýchod (1,1). Pro proudy platí $\text{offset} = 2/388$ a $\text{scale} = 384/388$; obdobně pro masku. Rastrové řádky jdou od severu, proto shader při vzorkování jednou obrací V.

Kód: `src/water/TileGrid.js`; `src/data/surface.js`; `src/render/TiledWaterRenderNode.js`

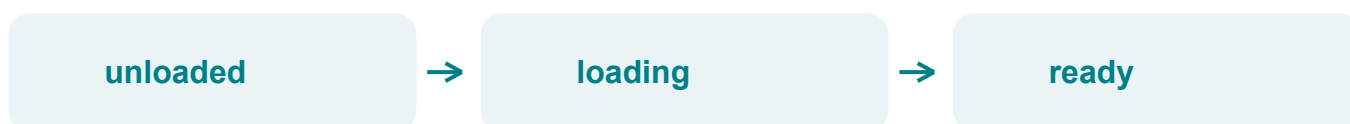
Výběr, fronta a životnost dlaždic

TileManager odděluje získávání dat od kreslení a hlídá, kolik dlaždic zůstává v paměti.

Výběr podle kamery

`selectCameraTiles()` použije `extent.view.visibleArea`, případně `view.extent`, ořízne jej rozsahem dat a najde indexy mřížky. Viditelné dlaždice mají přednost; uvnitř skupiny rozhoduje čtverec vzdálenosti od středu pohledu. Okolí přidá jeden prstenec přednačtení, celý seznam se omezí na 36 položek.

Watch sleduje polohu, heading, tilt, rozměry view a stationary. Změny slučuje časovačem 250 ms. Při širokém pohledu omezuje i samotné procházení mřížky a nastaví `limited`. Není zde víceúrovňové LOD.



Chyba → error → další pokus; zrušení → unloaded

Fronta a opakování

Ve výchozím nastavení běží dvě přípravy současně. Změna výběru zruší nepotřebné požadavky přes `AbortController`. Neúspěšné načtení má nejvýše tři pokusy celkem; prodleva je $2000 \cdot 2^{\text{attempts}}$ ms, tedy po prvním pokusu 4 s, po druhém 8 s. Kontrolní timer běží každé 2 s.

Uvolňování paměti

Nepotřebné záznamy se řadí podle `lastUsed` a odstraňují při překročení 160 MiB, 96 záznamů nebo po 30 s nevyužití. Požadované dlaždice se tímto krokem nevyhazují; pokud nový výsledek nelze vměstnat, vznikne `TILE_MEMORY_BUDGET`. GPU prostředky uvolní `RenderNode`.

Kód: `src/water/cameraTiles.js`; `src/water/TileManager.js`; `src/public/defaults.js`

Co provede RenderNode v jednom snímku

TiledWaterRenderNode je vlastní uzel ArcGIS renderovacího grafu: přijímá a vrací composite-color.

1. Čas a úklid	Zpracuje čekající odstranění GPU dat. Když není pauza, přičte <code>min(deltaTime, 0.1 s)</code> . Tím omezí skok času po prodlevě.
2. Program a upload	Při potřebě zkompile water program. Za snímek nahraje nejvýše jednu novou požadovanou dlaždici. V ustáleném stavu nic znovu nenahrává.
3. Stav WebGL	Zapne <code>DEPTH_TEST</code> s <code>LEQUAL</code> , vypne zápis hloubky i <code>CULL_FACE</code> . Pro barvy použije <code>SRC_ALPHA / ONE_MINUS_SRC_ALPHA</code> .
4. Společné uniforms	Předá projekci, čas, světlo, barvu, rychlost a parametry vln. Vektory east/north/up i světlo převede do prostoru kamery.
5. Každá aktivní dlaždice	Doplní model-view o počátek dlaždice, nastaví UV transformace a <code>worldOffset</code> , naváže VAO a dvě textury. Volá <code>drawElements(TRIANGLES, ..., UNSIGNED_SHORT, 0)</code> .
6. Další snímek	<code>requestRender()</code> pokračuje při animaci, čekajících uploadech nebo postupném zobrazení. Nakonec obnoví WebGL stav pro ArcGIS.

Vertex shader zachová pevnou geometrii

```
vUV = aUV;
vec4 position = uModelView * vec4(aPosition, 1.0);
vViewPosition = position.xyz;
gl_Position = uProjection * position;
```

Žádný posun vrcholů podle času zde není. Pro novou dlaždici používá průchod `fade = min(1, (now - readyAt)/450 ms)`, který násobí výslednou alfu. Jedna aktivní dlaždice odpovídá jednomu volání `drawElements`.

Kód: `src/render/TiledWaterRenderNode.js`; `src/render/shaders/velocity.vert.glsl`

Proudové pole řídí sklon a barvu

water.frag.glsl obsahuje procedurální kresbu; field.glsl se při sestavení programu vloží do jeho zdrojového textu.

1. Maska a vektor. Pokrytí wet < 0.01 pixel zahodí. fieldAt() nejprve ověří platnost nejbližšího vzorku a potom bilineárně smíchá čtyři RGBA hodnoty. Dělení výslednou alfa přenormuje platné sousedy. Další průměr platnosti čtyř okolních texelů utlumí proud u NoData.

```
// Zápis výpočtu, zkrácený pro vysvětlení:  
p = uWorldOffset + vUV * uSizeMeters;  
phaseA = fract(time / period);  
phaseB = fract(time / period + 0.5);  
weightA = pow(sin(PI * phaseA), 2.0);  
travel = velocity * speed * speedFactor * period;  
a = structure((p - travel * phaseA) / scale + seed);  
b = structure((p - travel * phaseB) / scale + seed);  
slope = mix(b, a, weightA);
```

2. Dvě měřítka detailu. Hrubá struktura má scale 7,5 a periodu 4,8 s, jemná 1,15 a 3,7 s; jemná používá speedFactor 1,13. speed je $0,12 \times \text{flowSpeed}$. Gradient šumu vzniká analyticky. structure() kombinuje tři vzorky podél proudu a potlačí část sklonu ve směru proudění.

3. Normála. Hrubý a jemný sklon mají síly 0,50 a 0,20. Jejich součet se násobí $\text{mix}(0.45, 1, \text{activity})$, kde $\text{activity} = \text{smoothstep}(0, \text{magnitudeReference}, |\text{velocity}|)$. Normála je $\text{normalize}(\text{up} - \text{slope.x} \cdot \text{east} - \text{slope.y} \cdot \text{north})$. Derivace $dFdx/dFdy$ potlačují příliš jemný detail v dálce.

4. Osvětlení. Fresnelův člen je $0,02 + 0,98 \cdot (1 - \max(N \cdot V, 0))^5$. Míchá tělovou barvu s konstantním nádechem oblohy. Odlesk používá poloviční vektor $H = \text{normalize}(L+V)$ a exponent mezi 100 a 48. Jas násobí celou barvu; nad 0,8 se světla měkce komprimují. Alfa = $\text{opacity} \times \text{wet} \times \text{fade}$.

Kód: src/render/shaders/water.frag.glsl; field.glsl; src/water/parameters.js

API, diagnostika a sestavení

Veřejné rozhraní skrývá dlaždice a WebGL. Aplikace spravuje kameru, zdrojové vrstvy a vlastní uživatelské rozhraní.

```
// Po získání instance water:
water.setParameters({ flowSpeed: 2, reflection: 1.3 });
water.setParameters({ paused: true });
const status = water.getStatus();
const off = water.on('error', error => {
  console.error(error.code, error.message);
});
// Při ukončení vlastní integrace, před zrušením view:
off();
water.destroy();
```

Diagnostika: `getStatus()` vrací stav, `ready`, `limited`, počty dlaždic, odhad CPU/GPU paměti, dotazy, `uploady`, zásahy `cache` a poslední chybu. Událost `statuschange` se posílá při významné změně, nikoli v každém snímku. `setParameters()` nejprve validuje celý patch; neplatné hodnoty neaplikuje.

Životní cyklus: `stop()` zruší dotazy, `watchery`, `cache` i GPU prostředky. Následující `start()` vše načte znovu a resetuje čas; vzhledové parametry zůstanou. `destroy()` je definitivní. Generační čítač zabraňuje přijetí opožděných výsledků z předchozího běhu.

Jak vzniknou soubory k nasazení

```
npm run build:lib      # knihovna do dist/
npm run build:examples # webové demo do demo-dist/
npm test              # jednotkové / veřejné API kontroly
npm run test:types     # deklarace TypeScript
npm run test:browser   # integrace v Chrome
```

Knihovna je ESM a ponechává `@arcgis/core` jako externí závislost (testovaná verze 5.1.26). GLSL se vkládá do JavaScriptu. Demo má samostatný build: na web patří `demo-dist/index.html` a `assets/`. Pro podsložku je nutné nastavit `--base`. Uvedené testy existují v repozitáři; při této úpravě dokumentace nebyly znovu spouštěny.

Kód: `src/public/FloodWaterRenderer.js`; `parameters.js`; `vite.lib.config.js`; `package.json`; `tests/`